

Случайные леса

В этом задании вам предстоит реализовать ансамбль деревьев решений, известный как случайный лес, применить его к публичным данным пользователей социальной сети Вконтакте, и сравнить его эффективность с бустингом, предоставляемым библиотекой `CatBoost`.

В результате мы сможем определить, какие подписки пользователей больше всего влияют на определение возраста и пола человека.

```
In [1]: import inspect
import random
from collections import Counter
from dataclasses import dataclass
from itertools import product
from typing import Callable, List, Tuple, Union, Optional, Any

import numpy as np
import scipy
import numpy.typing as npt
import pandas
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split
from tqdm.notebook import tqdm
```

```
In [2]: def set_seed(seed=42):
    np.random.seed(seed)
    random.seed(seed)

# Этой функцией будут помечены все места, которые необходимо дозаполнить
# Это могут быть как целые функции, так и отдельные части внутри них
# Всегда можно воспользоваться интроспекцией и найти места использования этой функции :)
def todo():
    stack = inspect.stack()
    caller_frame = stack[1]
    function_name = caller_frame.function
    line_number = caller_frame.lineno
    raise NotImplementedError(f"TODO at {function_name}, line {line_number}")

SEED = 0xC0FFEE
set_seed(SEED)
```

```
In [3]: def mode(data):
    counts = Counter(data)
    return counts.most_common(n=1)[0][0]
```

Задание 1 (2 балла)

Random Forest состоит из деревьев решений. Каждое такое дерево строится на одной из выборок, полученных при помощи bootstrap. Элементы, которые не вошли в новую обучающую выборку, образуют **out-of-bag** выборку. Кроме того, в каждом узле дерева мы случайным образом выбираем набор из `max_features` и ищем признак для предиката разбиения только в этом наборе.

Сегодня мы будем работать только с бинарными признаками, поэтому нет необходимости выбирать значение признака для разбиения.

Методы

`predict(X)` - возвращает предсказанные метки для элементов выборки `X`

Параметры конструктора

X , y - обучающая выборка и соответствующие ей метки классов. Из нее нужно получить выборку для построения дерева при помощи bootstrap. Out-of-bag выборку нужно запомнить, она понадобится потом.

`criterion="gini"` - задает критерий, который будет использоваться при построении дерева. Возможные значения: "gini" , "entropy" .

`max_depth=None` - ограничение глубины дерева. Если `None` - глубина не ограничена

`min_samples_leaf=1` - минимальное количество элементов в каждом листе дерева.

`max_features="auto"` - количество признаков, которые могут использоваться в узле. Если "auto" - равно `sqrt(X.shape[1])`

```
In [4]: def calculate_proportions(x: npt.ArrayLike) -> np.ndarray:
    x = np.asarray(x)
    if x.size == 0:
        return np.array([])

    _, counts = np.unique(x, return_counts=True)

    proportions = counts / x.size
    return proportions

def gini(x: npt.ArrayLike) -> float:
    """
    Calculate the Gini impurity of a list or array of class labels.

    Args:
        x (ArrayLike): Array-like object containing class labels.

    Returns:
        float: Gini impurity value.
    """

    proportions = calculate_proportions(x)

    if proportions.size == 0:
        return 0.0

    gini_impurity = np.sum(proportions * (1 - proportions))

    return float(gini_impurity)

def entropy(x: npt.ArrayLike) -> float:
    """
    Calculate the entropy of a list or array of class labels.

    Args:
        x (ArrayLike): Array-like object containing class labels.

    Returns:
        float: Entropy value.
    """

    proportions = calculate_proportions(x)

    if proportions.size == 0 or proportions.size == 1:
        return 0.0
```

```

non_zero_proportions = proportions[proportions > 0]

entropy_val = -np.sum(non_zero_proportions * np.log2(non_zero_proportions))

return float(entropy_val)

def gain(left_y: npt.ArrayLike, right_y: npt.ArrayLike, criterion: Callable[[npt.ArrayLike], float]) -> float:
    """
    Calculate the information gain of a split using a specified criterion.

    Args:
        left_y (ArrayLike): Class labels for the left split.
        right_y (ArrayLike): Class labels for the right split.
        criterion (Callable): Function to calculate impurity (e.g., gini or entropy).

    Returns:
        float: Information gain from the split.
    """

    R = np.concatenate((left_y, right_y), axis=None)
    HRL = left_y.shape[0] / R.shape[0] * criterion(left_y)
    HRr = right_y.shape[0] / R.shape[0] * criterion(right_y)
    return criterion(R) - HRL - HRr

```

```

In [5]: @dataclass
class DecisionTreeLeaf:
    classes: np.ndarray

    def __post_init__(self):
        self.max_class = mode(self.classes)

@dataclass
class DecisionTreeInternalNode:
    split_dim: int
    split_value: float
    left: Union["DecisionTreeInternalNode", DecisionTreeLeaf]
    right: Union["DecisionTreeInternalNode", DecisionTreeLeaf]

DecisionTreeNode = Union[DecisionTreeInternalNode, DecisionTreeLeaf]

```

```

In [6]: class DecisionTree:
    def __init__(self, X, y, X_oob, y_oob, criterion="gini", max_depth=None, min_samples_leaf=1, max_features="auto"):
        if criterion == "gini":
            self._criterion_func = gini
        elif criterion == "entropy":
            self._criterion_func = entropy
        else:
            raise ValueError(f"Unknown criterion: {criterion}. Use 'gini' or 'entropy'.")

        self._X = X
        self._y = y
        self._out_of_bag_X = X_oob
        self._out_of_bag_y = y_oob
        self._max_depth = max_depth
        self._min_samples_leaf = min_samples_leaf
        self._max_features = max_features

```

```

self._root = self._build_node(X, y, 0)

@property
def out_of_bag(self) -> Tuple[np.ndarray, np.ndarray]:
    return self._out_of_bag_X, self._out_of_bag_y

def _get_feature_subset(self, n_total_features: int) -> np.ndarray:
    if isinstance(self._max_features, int):
        if self._max_features >= n_total_features:
            return np.arange(n_total_features)
        else:
            return np.random.choice(n_total_features, self.max_features, replace=False)
    elif isinstance(self._max_features, float):
        if self._max_features >= 1.0:
            return np.arange(n_total_features)
        else:
            n_selected = max(1, int(self.max_features * n_total_features))
            return np.random.choice(n_total_features, n_selected, replace=False)
    elif self._max_features in ["auto", "sqrt"]:
        n_selected = max(1, int(np.sqrt(n_total_features)))
        return np.random.choice(n_total_features, n_selected, replace=False)
    elif self._max_features == "log2":
        n_selected = max(1, int(np.log2(n_total_features)))
        return np.random.choice(n_total_features, n_selected, replace=False)
    else:
        raise ValueError(f"Invalid max_features value: {self.max_features}")

def _best_split(self, X: np.ndarray, y: np.ndarray) -> Tuple[Optional[int], Optional[float], float]:
    n_samples, n_features = X.shape
    best_gain = -1.0
    best_split_dim = None
    best_split_value = None

    feature_indices = self._get_feature_subset(n_features)

    for dim in feature_indices:
        feature_values = X[:, dim]
        unique_values = np.unique(feature_values)

        if unique_values.size > 1:
            splits = (unique_values[:-1] + unique_values[1:]) / 2.0
        else:
            splits = []

        for split_value in splits:
            left_indices = feature_values <= split_value
            right_indices = feature_values > split_value

            if np.sum(left_indices) < self._min_samples_leaf or np.sum(right_indices) < self._min_samples_leaf:
                continue

            left_y = y[left_indices]
            right_y = y[right_indices]

            current_gain = gain(left_y, right_y, self._criterion_func)

            if current_gain > best_gain:
                best_gain = current_gain
                best_split_dim = dim
                best_split_value = split_value

    return best_split_dim, best_split_value, best_gain

```

```

def _build_node(self, X: np.ndarray, y: np.ndarray, depth: int) -> DecisionTreeNode:
    n_samples = X.shape[0]

    if np.unique(y).size == 1:
        return DecisionTreeLeaf(classes=y)

    if self._max_depth is not None and depth >= self._max_depth:
        return DecisionTreeLeaf(classes=y)

    if n_samples < self._min_samples_leaf * 2:
        return DecisionTreeLeaf(classes=y)

    best_dim, best_value, best_gain = self._best_split(X, y)

    if best_gain <= 0 or best_dim is None:
        return DecisionTreeLeaf(classes=y)

    left_indices = X[:, best_dim] <= best_value
    right_indices = X[:, best_dim] > best_value

    left_X, left_y = X[left_indices], y[left_indices]
    right_X, right_y = X[right_indices], y[right_indices]

    left_child = self._build_node(left_X, left_y, depth + 1)
    right_child = self._build_node(right_X, right_y, depth + 1)

    return DecisionTreeInternalNode(
        split_dim=best_dim,
        split_value=best_value,
        left=left_child,
        right=right_child
    )

def _traverse_tree(self, x: np.ndarray, node: DecisionTreeNode) -> Any:
    if isinstance(node, DecisionTreeLeaf):
        return node.max_class

    if x[node.split_dim] <= node.split_value:
        return self._traverse_tree(x, node.left)
    else:
        return self._traverse_tree(x, node.right)

def _predict(self, X: np.ndarray, node: DecisionTreeNode) -> np.ndarray:
    if X.ndim == 1:
        X = X.reshape(1, -1)

    preds = np.array([self._traverse_tree(x, node) for x in X])
    return preds

def predict(self, X: np.ndarray) -> np.ndarray:
    return self._predict(X, self._root)

```

Задание 2 (2 балла)

Теперь реализуем сам Random Forest. Идея очень простая: строим n деревьев, а затем берем модальное предсказание.

Параметры конструктора

`n_estimators` - количество используемых для предсказания деревьев.

Остальное - параметры деревьев.

Методы

`fit(X, y)` - строит `n_estimators` деревьев по выборке `X`.

`predict(X)` - для каждого элемента выборки `X` возвращает самый частый класс, который предсказывают для него деревья.

```
In [7]: class RandomForestClassifier:

    _n_features: int = None

    def __init__(self, criterion="gini", max_depth=None, min_samples_leaf=1, max_features="auto", n_estimators=10):
        self._criterion = criterion
        self._max_depth = max_depth
        self._min_samples_leaf = min_samples_leaf
        self._max_features = max_features
        self._n_estimators = n_estimators
        self._estimators = []

    @property
    def estimators(self) -> List[DecisionTree]:
        return self._estimators

    @property
    def n_features(self) -> int:
        if self._n_features is None:
            raise RuntimeError("Fit random forest before accessing to number of features properties")
        return self._n_features

    def fit(self, X, y):
        self._n_samples, self._n_features = X.shape
        for i in range(self._n_estimators):

            all_indices = np.arange(self._n_samples)
            bootstrap_indices = np.random.choice(self._n_samples, size=self._n_samples, replace=True)
            oob_indices = np.setdiff1d(all_indices, np.unique(bootstrap_indices), assume_unique=True)

            X_bootstrap = X[bootstrap_indices]
            y_bootstrap = y[bootstrap_indices]
            X_oob = X[oob_indices]
            y_oob = y[oob_indices]

            self._estimators.append(
                DecisionTree(
                    X_bootstrap, y_bootstrap, X_oob, y_oob, self._criterion, self._max_depth, self._min_samples_leaf, self._max_features
                )
            )

    def predict(self, X):
        predicts = np.array([tree.predict(X) for tree in self._estimators])
        output = []
        for tree_preds in predicts.T:
            output.append(mode(tree_preds))
        return output
```

Задание 3 (2 балла)

Часто хочется понимать, насколько большую роль играет тот или иной признак для предсказания класса объекта. Есть различные способы посчитать его важность. Один из простых способов

сделать это для Random Forest - посчитать out-of-bag ошибку предсказания `err_oob` , а затем перемешать значения признака `j` и посчитать ее (`err_oob_j`) еще раз. Оценкой важности признака `j` для одного дерева будет разность `err_oob_j` - `err_oob` , важность для всего леса считается как среднее значение важности по деревьям.

Реализуйте функцию `feature_importance` , которая принимает на вход Random Forest и возвращает массив, в котором содержится важность для каждого признака.

```
In [8]: def accuracy_score(y_true: np.ndarray, y_pred: np.ndarray) -> float:
        y_true = y_true.reshape(-1)
        y_pred = y_pred.reshape(-1)
        return np.mean(y_true == y_pred)

def feature_importance(rfc: RandomForestClassifier) -> np.ndarray:
    if not rfc.estimators:
        raise RuntimeError("The forest must be fitted before calculating feature importance.")

    n_features = rfc.n_features
    importances = np.zeros(n_features)
    n_oob_trees = 0

    for tree in rfc.estimators:
        X_oob, y_oob = tree.out_of_bag

        if X_oob is None or y_oob is None or X_oob.shape[0] == 0:
            continue

        n_oob_trees += 1

        y_pred_oob_baseline = tree.predict(X_oob)
        baseline_accuracy = accuracy_score(y_oob, y_pred_oob_baseline)

        for j in range(n_features):
            X_oob_permuted = X_oob.copy()
            np.random.shuffle(X_oob_permuted[:, j])

            y_pred_oob_permuted = tree.predict(X_oob_permuted)
            permuted_accuracy = accuracy_score(y_oob, y_pred_oob_permuted)
            importances[j] += baseline_accuracy - permuted_accuracy

        if n_oob_trees > 0:
            importances /= n_oob_trees
        else:
            print("Warning: No trees with OOB samples found. Feature importances might be unreliable.")

    importances[importances < 0] = 0

    total_importance = np.sum(importances)
    if total_importance > 0:
        importances /= total_importance

    return importances

def most_important_features(importance, names, k=20):
    indices = np.argsort(importance)[::-1][:k]
    return np.array(names)[indices]
```

Наконец, пришло время протестировать наше дерево на простом синтетическом наборе данных. В результате точность должна быть примерно равна `1.0` , наибольшее значение важности должно быть у признака с индексом `4` , признаки с индексами `2` и `3` должны быть одинаково важны, а остальные признаки - не важны совсем.

```
In [9]: def synthetic_dataset(size):
        X = [
            (np.random.randint(0, 2), np.random.randint(0, 2), i % 6 == 3, i % 6 == 0, i % 3 == 2, np.random.randint(0, 2))
            for i in range(size)
        ]
        y = [i % 3 for i in range(size)]
        return np.array(X), np.array(y)

X, y = synthetic_dataset(1000)
rfc = RandomForestClassifier(n_estimators=100)
rfc.fit(X, y)
print("Accuracy:", np.mean(rfc.predict(X) == y))
print("Importance:", feature_importance(rfc))
```

```
Accuracy: 1.0
Importance: [0.          0.          0.23285442 0.2442336  0.52291198 0.          ]
```

Задание 4 (1 балл)

Теперь поработаем с реальными данными.

Выборка состоит из публичных анонимизированных данных пользователей социальной сети Вконтакте. Первые два столбца отражают возрастную группу (`zoomer` , `doomer` и `boomer`) и пол (`female` , `male`). Все остальные столбцы являются бинарными признаками, каждый из них определяет, подписан ли пользователь на определенную группу/публичную страницу или нет.

Необходимо обучить два классификатора, один из которых определяет возрастную группу, а второй - пол.

Эксперименты с множеством используемых признаков и подбор гиперпараметров приветствуются. Лес должен строиться за какое-то разумное время.

```
In [10]: def read_dataset(path):
        dataframe = pandas.read_csv(path, header=0)
        dataset = dataframe.values.tolist()
        random.shuffle(dataset)
        y_age = [row[0] for row in dataset]
        y_sex = [row[1] for row in dataset]
        X = [row[2:] for row in dataset]

        return np.array(X), np.array(y_age), np.array(y_sex), list(dataframe.columns)[2:]
```

```
In [11]: X, y_age, y_sex, features = read_dataset("vk.csv")
X_train, X_test, y_age_train, y_age_test, y_sex_train, y_sex_test = train_test_split(X, y_age, y_sex, train_size=0.9)
```

Возраст

```
In [12]: rfc = RandomForestClassifier(n_estimators=10)

rfc.fit(X_train, y_age_train)
print("Accuracy:", np.mean(rfc.predict(X_test) == y_age_test))
print("Most important features:")
for i, name in enumerate(most_important_features(feature_importance(rfc), features, 20)):
    print(str(i + 1) + ".", name)
```


Accuracy: 0.7049180327868853

Most important features:

1. ovsyanochan
2. i_d_t
3. 4ch
4. mudakoff
5. bot_maxim
6. rhymes
7. xfilm
8. bratishkinoff
9. styd.pozor
10. leprum
11. thesmolny
12. iwantyou
13. reflexia_our_feelings
14. privetuyeba
15. dayvinchik
16. bog_memes
17. ebashproklatie
18. ohhluul
19. tumblr_vacuum
20. pravdashowtop

Пол

```
In [13]: rfc = RandomForestClassifier(n_estimators=10)
rfc.fit(X_train, y_sex_train)
print("Accuracy:", np.mean(rfc.predict(X_test) == y_sex_test))
print("Most important features:")
for i, name in enumerate(most_important_features(feature_importance(rfc), features, 20)):
    print(str(i + 1) + ". ", name)
```

Accuracy: 0.8436317780580076

Most important features:

1. 4ch
2. 40kg
3. igm
4. sh.cook
5. i_d_t
6. cook_good
7. be.beauty
8. be.women
9. mudakoff
10. femalememe
11. academyofman
12. girlmeme
13. modnailru
14. rapnewrap
15. zerofat
16. overhear
17. woman.blog
18. recipes40kg
19. h.made
20. thesmolny

CatBoost

В качестве альтернативы попробуем CatBoost.

Установить его можно просто с помощью `pip install catboost`. Тutorials можно найти, например, [здесь](#) и [здесь](#). Главное - не забудьте использовать `loss_function='MultiClass'`.

Сначала протестируйте CatBoost на синтетических данных. Выведите точность и важность признаков.

```
In [14]: X, y = synthetic_dataset(1000)

cb_model = CatBoostClassifier(iterations=10,
                             learning_rate=1,
                             depth=2,
                             loss_function='MultiClass')

cb_model.fit(X, y)
y_pred = cb_model.predict(X)

print("Accuracy:", accuracy_score(y_pred, y))
print("Importance:", cb_model.feature_importances_)
```

```
0:      learn: 0.4239784      total: 46.7ms   remaining: 420ms
1:      learn: 0.1037410      total: 47.1ms   remaining: 188ms
2:      learn: 0.0475388      total: 47.4ms   remaining: 111ms
3:      learn: 0.0271089      total: 47.7ms   remaining: 71.5ms
4:      learn: 0.0157713      total: 48.1ms   remaining: 48.1ms
5:      learn: 0.0102860      total: 48.4ms   remaining: 32.3ms
6:      learn: 0.0074490      total: 48.7ms   remaining: 20.9ms
7:      learn: 0.0051865      total: 49ms      remaining: 12.3ms
8:      learn: 0.0041190      total: 49.3ms   remaining: 5.48ms
9:      learn: 0.0032379      total: 49.6ms   remaining: 0us
```

Accuracy: 1.0

Importance: [0. 0. 22.24583462 28.33697139 49.41719399 0.]

Задание 5 (3 балла)

Попробуем применить один из используемых на практике алгоритмов. В этом нам поможет CatBoost. Также, как и реализованный ими RandomForest, применим его для определения пола и возраста пользователей сети Вконтакте, выведите названия наиболее важных признаков так же, как в задании 3.

Эксперименты с множеством используемых признаков и подбор гиперпараметров приветствуются.

```
In [15]: X, y_age, y_sex, features = read_dataset("vk.csv")
X_train, X_test, y_age_train, y_age_test, y_sex_train, y_sex_test = train_test_split(X, y_age, y_sex, train_size=0.9)
X_train, X_eval, y_age_train, y_age_eval, y_sex_train, y_sex_eval = train_test_split(
    X_train, y_age_train, y_sex_train, train_size=0.8
)
```

```
In [16]: max_depth = range(1, 10, 3)
min_samples_leaf = range(1, 10, 3)
learning_rate = np.linspace(0.001, 1.0, 5)

def get_best_params(y_train, y_eval):
    best_score, best_params = None, None
    for lr, md, msl in list(product(learning_rate, max_depth, min_samples_leaf)):
        cb_model = CatBoostClassifier(learning_rate=lr,
                                      max_depth=md,
                                      min_data_in_leaf=msl,
                                      loss_function='MultiClass',
                                      verbose=None,
                                      logging_level='Silent')

        cb_model.fit(X_train, y_train)
        y_pred = cb_model.predict(X_eval)
        acc = accuracy_score(y_pred, y_eval)
        if not best_score:
            best_score = acc
```

```

        best_params = [lr, md, msl]
        if acc > best_score:
            best_score = acc
            best_params = [lr, md, msl]

    return best_params, best_score

```

Возраст

```

In [17]: best_params, best_score = get_best_params(y_age_train, y_age_eval)
best_params, best_score

```

```

Out[17]: ([np.float64(0.25075), 7, 1], np.float64(0.7435178696566223))

```

```

In [18]: lr, md, msl = best_params
cb_model = CatBoostClassifier(learning_rate=lr,
                             max_depth=md,
                             min_data_in_leaf=msl,
                             loss_function='MultiClass',
                             verbose=None,
                             logging_level='Silent')

cb_model.fit(X_train, y_age_train)
y_pred = cb_model.predict(X_test)

print("Accuracy:", accuracy_score(y_age_test, y_pred))
print("Most important features:")
for i, name in enumerate(mostImportantFeatures(cb_model.feature_importances_, features, 10)):
    print(str(i + 1) + ". ", name)

```

Accuracy: 0.7452711223203027

Most important features:

1. mudakoff
2. 4ch
3. dayvinchik
4. ovsyanochan
5. rapnewrap
6. styd.pozor
7. kino_mania
8. rhymes
9. exclusive_muzic
10. leprum

Пол

```

In [19]: best_params, best_score = get_best_params(y_sex_train, y_sex_eval)
best_params, best_score

```

```

Out[19]: ([np.float64(0.25075), 4, 1], np.float64(0.8514365802382621))

```

```

In [20]: lr, md, msl = best_params
cb_model = CatBoostClassifier(learning_rate=lr,
                             max_depth=md,
                             min_data_in_leaf=msl,
                             loss_function='MultiClass',
                             verbose=None,
                             logging_level='Silent')

cb_model.fit(X_train, y_sex_train)
y_pred = cb_model.predict(X_test)

print("Accuracy:", accuracy_score(y_sex_test, y_pred))

```

```
print("Most important features:")
for i, name in enumerate(most_important_features(cb_model.feature_importances_, features, 10)):
    print(str(i + 1) + ".", name)
```

Accuracy: 0.8663303909205549

Most important features:

1. 40kg
2. mudakoff
3. girlmeme
4. modnailru
5. i_d_t
6. 9o_6o_9o
7. be.beauty
8. thesmolny
9. zerofat
10. femalemem

In []: